Errata and model clarifications

Pure Infinitely Self-Modifying Code is Realizable and Turing-complete

Gregory Morse

International Journal of Electronics and Telecommunications, 64(2), 123–129 (2018). DOI: 10.24425/119359.

The following local corrections specify the intended primitives and model. Page numbers refer to the published article.

M01. Listing 2, p.124: one mnemonic in DEFADDR. Replace only its third instruction, `mov acmltr, [acmltr]`, by `xchg acmltr, [acmltr]`.
The following `MOFFSMOV(labelname, addr)` remains as printed. Both accessed words must be wholly contained in code. Restore any temporary opcode/address writes before fetching the affected instructions.

M02. Sections I–II, p.123: write purity and the unbounded model.
In the definition of infinitely self-modifying code, replace "always modifies code instructions in memory in every statement" by "writes to code memory in every executed computational instruction". An architectural write counts even when the stored value is unchanged. After Section II's first paragraph, insert:
Unboundedness and Turing-completeness refer to an idealized machine with unbounded memory and addresses; arbitrarily large addresses may be represented by finite variable-length operands. The x86 and x86-64 listings give bounded realizations of this model.

M03. Prose after Listing 8, p.127: execution property. Replace the opening clause "The machine executes every single instruction at least once" by "The machine writes to an instruction field in every executed computational instruction". Retain the remainder of the sentence.

M04. Listings 2 and 5, pp.124–125: operand-width contract.
After Listing 2, insert:
OP64 and USEWORDOP64 apply operations to corresponding 32-bit halves. Their MOV, XOR, AND, OR and NOT forms have the corresponding whole-word meaning; in OP64's immediate form, data is replicated in both halves. Independent half-word ADD or SUB gives whole-word arithmetic only when carry or borrow is absent; otherwise it must be propagated. DWORD helpers restore EAX; in 64-bit mode their final RAX is the zero extension of EAX. Full RAX restoration assumes a zero-extended entry RAX.

M05. Listing 10, pp.128–129: independent branch target.
Change the interface to `DoBranch(x, y, target, not, gt, sign)`.
In `mov [lbl0+IMMOFFS], x-lbl2-OPSZ1-PTRSZ`, replace only `x` by `target`.
Add this precondition: `x`, `y`, `lbl0+IMMOFFS`, `lbl1+IMMOFFS` and `lbl2+OPSZ1` occupy pairwise disjoint complete instruction-operand words; the taken and fall-through entries use matching `JUMPTARGET` continuations. The bracketed CALL remains as printed, with MASM NEAR-code-label typing.

M06. Identifier spellings, pp.125, 127 and 129.
In Listing 3, change `sbb [maskptr], 0` to `sbb [datamaskptr], 0`. In Listing 8, change `IMMOFS` to `IMMOFFS`; in Listing 12, change `IMMOFSS` to `IMMOFFS`. These identify the declared comparison parameter and the constant in Table II.

M07. Listing 11, p.129: multiplication branch plumbing. Replace the delta write by:
`mov [lbl1+IMMOFFS], lbl9-lbl3-OPSZ1-PTRSZ`.
The first `JUMPTARGET()` becomes `JUMPTARGET(lblbody)`; the second becomes `JUMPTARGET(lbl9)`. Use distinct complete operand and scratch words and the existing code-backed stack contract.

M08. Listing 6, p.125: constant-multiplication bit gaps. Change "Leading" to "Trailing" in the definition of `NLZ`. Retain its identifier and the first shift; in the two later SHLs only, change `NLZ(val)` to `NLZ(val) + 1`. Treat `val` as an unsigned 32-bit pattern; signed multiplication retains its low 32-bit product. Compile-time logical shifts may consume all 32 bits. For `val=0` emit `mov SZW32 ptr [dataptr], 0`; omit an initial shift by zero. Use distinct complete code-operand words for `dataptr` and the final ADD immediate.

M09. Listing 6, p.125: DEFZERO immediate width. Use the existing force-32-bit constant and make the DWORD width explicit:
`labelname: xor SZW32 ptr [labelname + IMMOFFS], CNSTFRC32`.

M10. Listing 5, p.125: declared cursor fields. Replace all three `[sourceinit]` operands by `[source + DATAOFFS]`, and all three `[destinit]` operands by `[dest + DATAOFFS]`. Here `DATAOFFS=IMMOFFS` in 32-bit mode and zero in 64-bit mode. Require DF=0, complete disjoint source/destination regions and cursor words, and representable pointers and subtraction size. In Listing 6, p.126, use fresh cursor labels and expand its example to:
`TRANSFERDATA(srcptr, OFFSET codeblueprint, dstptr, OFFSET codedest, codesize)`.

M11. Listing 12, p.129: four substitutions (signed copy first).
`mov [lbl11+IMMOFFS], acmltr` →
`mov [lbl13+IMMOFFS], acmltr`;
`($-lbl19)-($-lbl10-OPSZ1-PTRSZ)` →
`lbl19-lbl10-OPSZ1-PTRSZ`;
`JUMPTARGET(lbl10)` → `JUMPTARGET(lbldivbody)`;
`mov [lbl10+IMMOFFS], 0` → `mov [lbl10+OPSZ1], 0`.
Require $y \neq 0$, distinct complete operand and scratch words, and the existing code-backed stack contract. Signed division truncates toward zero; remainder has the dividend sign. Exclude MIN_INT/−1 for a mathematical signed quotient. The counter spelling is already corrected by M06.

M12. Listing 2, p.125: reusable terminal marker. Replace `mov [$], 0` by `mov SZW32 ptr [$+IMMOFFS], 0`. Observe termination immediately after this marker; host stopping and output handling lie outside the pure computation.

M13. Listing 8, p.127: input-symbol source field. In the `USEWORD32` call writing `inputupdate+REGIMMOFFS`, replace only its source `curinputval` by `curinputval+IMMOFFS`.

M14. Same listing: scalar owner. Replace `DEFDATA(0, curtinputval)` by `DEFDATA32(curtinputval, 0)`.

M15. Same listing: comparison-mask copies. In both calls replace `inputvalmask, IMMOFFS` by `inputvalmask + IMMOFFS, 0`; copy that mask to both halves.

M16. Listings 7–8, pp.126–127: complete generalized input node. Use the generalized node of Fig.1 in Listing 8. Add `backptrval, fwdptrval` to DEFINPUT's arguments; after its symbol owner append
`DEFADDR(itapebptr&Index&, backptrval)` and
`DEFADDR(itapefptr&Index&, fwdptrval)`.
Initialize these values to the adjacent node entries. Insert `curttapeentry:` immediately before `DEFADDR(curttape, 0)`. In the first TRANSFERDATA, replace only `OPDBLARGSZ` by `curttapeentry - curitape`.

M17. Listing 8, p.127: transition entry and complete extent. Insert `curttapeend:` immediately after `DEFADDR(usettapeval, 0)`. In the second TRANSFERDATA replace its last two arguments:
`OFFSET curtape, TSZ` →
`OFFSET curttapeentry, curttapeend - curttapeentry`.

M18. Listing 8, p.127: DEFDATA cursor operands. Use M10's `DATAOFFS` at every read/write of `curitapeinit`, `curttapeinit`, `itapeptr` and `ttapeptrinit`: replace a bare cursor label by `label + DATAOFFS`. This includes both input exchanges, direction/head updates, failure comparison and resets. These labels denote a MOV entry on x86, an operand on x64.

M19. Same listing: transition-pointer packet fields and strides. Replace the pending pointer copy and cursor update by:
`USEWORDOP64(mov, ttapeupdate + IMMOFFS, OPDBLARGSZ, curttapeinit + DATAOFFS, PTRSZ32);`
`ttapeupdate: OP64(xor, curttapeinit + DATAOFFS, PTRSZ32, CNSTFRC32)`.
DWORD pointer halves are four bytes apart; DATAOP packets are ten bytes apart.

M20. Listing 9, p.127: accumulator-storage operands. Apply M10's `DATAOFFS` to all four memory operands: `[acmltrsave]` and `[acmltrptrsave]` become `[acmltrsave + DATAOFFS]` and `[acmltrptrsave + DATAOFFS]`, respectively. In the pointer declaration replace only `OFFSET acmltrsave` by `OFFSET acmltrsave + DATAOFFS`. The save declaration and instruction order are unchanged.

Justification of the corrections

The repaired storage primitive.
Let M be the initial word memory, l the operand field at `labelname`, $p = M(l)$, $q = M(p)$, and a the initial accumulator. The first exchange writes a to l and loads p; the following store restores $M(l) = p$. The corrected third instruction writes p to p and loads q. The final MOFFS store writes q back to p. Thus the complete macro satisfies

$$(\mathrm{acc}_{\mathrm{final}}, M_{\mathrm{final}}) = (M(M(l)), M).$$

For $p = l$, the temporary value at l is l and the final store restores it. Each reached instruction writes a code field when l and p are complete code fields. The original boundary effect is retained for unbounded, 32-bit and 64-bit words and finite repetitions; native uses require current-byte fetch.

Split-word operations.
For $B = 2^{32}$, write a 64-bit word as $u = u_L + Bu_H$ with $0 \le u_L, u_H < B$. Bitwise Boolean operations and corresponding-half assignment act independently on these two disjoint bit ranges. Therefore recombining their results gives the whole-word operation. Repeating an immediate d gives $d + Bd$, which also gives the expected full-word zero and all-ones masks. For addition, the high half is $u_H + v_H + c \pmod B$, where $c = \lfloor (u_L + v_L)/B \rfloor$; for subtraction it is $u_H - v_H - b \pmod B$, where $b = 1$ exactly when $u_L < v_L$.

The branch target.
Let $n =$ `lbl2` $+$ `OPSZ1` $+$ `PTRSZ` follow the CALL and let t be the independent target. The comparison restores inputs and AX, giving all ones when the selected relation holds, otherwise zero. The displacement is

$$d = \begin{cases} t - n, & \text{if the relation holds,} \\ 0, & \text{otherwise.} \end{cases}$$

$$n + d = \begin{cases} t, & \text{if the relation holds,} \\ n, & \text{otherwise.} \end{cases}$$

These are 32-bit modular equations. Disjoint complete words preserve the comparison; the existing stack and matching continuations restore SP and AX. The target substitution gives all six signed and unsigned conditional branches.

Unboundedness and execution purity.
Every address has a finite representation in the unbounded word model. The checked SMC simulation preserves TM halting/divergence and writes code at every step. An explicit nonhalting walker changes fields beyond every finite bound. Purity concerns reached instructions; storage owners need not all execute.

The multiplication correction.
With $n =$ `lbl3` $+ 1 + 4$ and $t =$ `lbl9`, the corrected delta is $t - n$ and its store addresses the AND's immediate field. The zero-test mask selects that delta exactly when the remaining multiplier is zero; otherwise the CALL displacement stays zero. Hence CALL reaches the existing exit or body continuation. The explicit labels name those same continuations. The body preserves $\mathrm{total} + \mathrm{factor} \cdot \mathrm{multiplier}$, using $y = 2\lfloor y/2 \rfloor + (y \bmod 2)$. Right shifts reduce the multiplier to zero after at most 32 body passes, giving the low 32-bit product. The operand exchanges and matching stack continuations restore their boundary registers. The shift-and-add algorithm is unchanged.

The constant-multiplication correction.
For nonzero v, let t count trailing zeros and $r = v/2^{t+1}$, so $v = 2^t(1 + 2r)$. The first shift selects the first bit; later shifts by $t + 1$ include the consumed bit. The work invariant is $\mathrm{total} + 2\,\mathrm{factor}\,v$ modulo 2^{32}. Compile-time v shrinks; at most 32 retained bits give the low-word product, including signed patterns. Zero uses one store. EAX is restored, only two code words are written, and SHL counts are 1–31.

The zeroing correction.
The corrected instruction has a complete 32-bit immediate at offset six. For its current operand word v, the destination and fetched immediate are both v, so it stores $v \oplus v = 0$. The header and exterior bytes are preserved; AX/SP are unchanged and CF is cleared. The same full-width packet remains reusable, for every initial word including zero.

The template-copy correction.
With DF=0, the MOVS widths sum to `size` and copy the original source bytes. Exchanges save and restore SI/DI, leaving $p +$ `size` in each cursor; subtraction restores p. Complete field/buffer separation preserves saved words and source. Only destination bytes change; DEFDATA prefix effects are retained. The scalar is the existing DEFDATA32 owner: initially zero at offset six, then preserving its live patched value, whole memory and AX/SP/CF. The input copy reads COMPARE's live imm32 at six and writes XOR's imm32 at three. Exchange/store/exchange restores the source and low AX, preserving CF, headers and exterior bytes.
The AND mask is its DWORD immediate at offset six; its start contains opcode and address bytes. Reading that same immediate for both halves copies zero or all ones, restoring the source and low accumulator after each copy.
The generalized node has a symbol and two neighbor owners; ten bytes leave the neighbors unchanged. The transition has two pointer and three DWORD owners. Copying to its operand label writes opcode bytes into the pointer. Entry-to-end copying initializes every field and complete packet.
The same DATAOFFS selects the restored cursor value in Listing 8. Bare x86 labels read opcode/address bytes; adding IMMOFFS to an x64 operand label crosses its end. The pending-pointer copy writes XOR immediates at offsets 6 and 16, preserving both headers. The later XORs target consecutive pointer halves; zero/all-ones masks select the old/new whole pointer.

The accumulator-storage operands.
On x86 a DEFDATA label names its ten-byte MOV header, with its full word at offset six. On x64 the label names the MOFFS word itself. DATAOFFS therefore selects exactly the save and pointer fields in both modes; the corrected pointer contains the save-field address. The emitted image and actual entry/body/three-instruction exit trace restore the full seed AX at the exit-code boundary, before the trailing declaration packets.

The division corrections.
The signed prefix initializes the divisor field subsequently normalized and read by the body. With $n =$ `lbl10` $+ 1 + 4$ and $t =$ `lbl19`, the corrected delta is $t - n$; the zero-counter guard selects exit, otherwise the existing body continuation at n. Its fresh name changes no instruction. The exit reset writes exactly the CALL's four-byte displacement at offset one, preserving its opcode and restoring zero for reuse. The restoring arithmetic maintains $p = qy + r$, $0 \le r < y$ for the processed dividend prefix p. Each new bit b forms $u = 2r + b$: take $(2q, u)$ if $u < y$, otherwise $(2q + 1, u - y)$. After 32 bits this gives quotient and remainder; each trial fits in 32 bits. Lean checks all four complete byte executions, result/register restoration, code-write purity and exterior-byte frames.

The terminal marker.
The ten-byte DWORD MOV writes its own four zero immediate bytes, preserves encoding, memory and AX/SP/CF, and advances to the observation point. This code write counts even when unchanged; the marker remains reusable.

Machine-checked evidence.
The proof ledger records the positive results and remaining realization conditions. Key Lean results (relative to `SmcTuring`): `Defaddr.repair_equivalent`; `Appendix.MultiplyByteCorrect.correct`; `Appendix.ConstantWords.contract`; `Appendix.ZeroImage.correct`; `Native.StringFields.effect`; `Appendix.InputDword.copy_correct`; `Native.DwordOwner.correct`; `Appendix.Listing8MaskCopy.broadcast_correct`; `Appendix.Listing8PrototypeCopy.node_copy`; `Appendix.Listing8PrototypeCopy.transition_copy`; `Appendix.Listing8CursorFields.source_cursor`; `Appendix.Listing8PointerTargets.immediate_patch`; `Appendix.Listing9Fields.corrected_pointer`; `Appendix.Listing9ExecutionImage.execute`; `Native.TerminalMarker.correct`; `Appendix.DivisionByteCorrect.correct`; `SMC.Concrete.pure_unbounded_smc_witness`.